

When Does Adapting an AQM's Parameters Pay? A Scaling Law in `target`/RTT, Measured on Stock Linux

Amritha S, Yugeshwaran P, Deepti Annuncia
Department of Electronics and Communication Engineering
School of Electronics Engineering (SENSE)
Vellore Institute of Technology, Chennai, India

Abstract—Linux's default queue discipline, `fq_codel`, exposes four parameters fixed at configuration time. Whether adapting them at runtime is worth doing has been answered both ways in the literature, with systems reporting gains and evaluations reporting none. We show that both answers are right, and that which one applies is decided by a single dimensionless number.

We build a three-node namespace testbed in which the bottleneck sits on the data path, add eBPF flow telemetry at the `tc` egress hook, and implement a userspace controller adapting all four parameters by the AIMD policy of Adaptive RED under a gradient-based trajectory estimate. Across 178 runs spanning three workloads, eight alternative queue disciplines and path RTTs from 2 to 200 ms, we find that the benefit of adaptation is governed not by path RTT but by the ratio $r = \text{target}/\text{RTT}$, saturating at 20.5% with half the benefit at $r = 0.50$ ($R^2 = 0.986$).

The hypothesis, its three predictions and their numerical thresholds were committed to the repository before the campaign that tests them ran. Ratio invariance, the prediction able to refute the claim, holds to 2.0 percentage points across a sixteenfold range of RTT while target, interval, control period and run duration are scaled together. Held-out interior ratios are predicted to 1.4 percentage points. At a fixed 20 ms path the benefit is 1.4% and not significant at $r = 0.25$ against 17.6% and significant at $r = 1.0$: same path, twelve times the benefit, which no RTT-based explanation reproduces.

The consequence is a predicate an operator can evaluate in advance. CoDel's own guidance places a correctly configured deployment at $r \approx 0.05$ to 0.10, so half the available benefit arrives five to ten times outside that envelope. Adaptation is a remedy for misconfiguration rather than an improvement on correct configuration, which unifies the null results in this work and in the literature as one effect. The gain is bought with roughly 63% more drops and comparable retransmissions, at a throughput cost never exceeding -0.4% . Four defects in our own controller, each an absolute constant in a system whose subject is a dimensionless ratio, would have decided this result rather than measured it; we report them and the checks that caught them.

Index Terms—active queue management, bufferbloat, `fq_codel`, eBPF, traffic control, Linux, reproducibility

I. INTRODUCTION

Bufferbloat – persistently full buffers inflating queueing delay without affecting throughput – was named by Gettys and Nichols over a decade ago [1], and the Linux response has largely converged on `fq_codel` [6], now the default queue discipline on most distributions. `fq_codel` combines

Deficit Round Robin scheduling across roughly a thousand flow queues with an independent CoDel [4], [5] instance per queue.

`fq_codel` exposes four parameters that do not change once set: `target` (5 ms), `interval` (100 ms), `limit` (10240 packets) and `quantum` (1514 bytes). It is natural to ask whether these should adapt to load, and there is precedent: Adaptive RED [3] adapts RED's $\max_p$ by AIMD against a queue-length target, and Ye and Leung [15] derive stability conditions for CoDel and adapt its `interval` analytically.

This paper asks that question and takes the answer seriously in both directions. We implement a controller that adapts all four parameters, and we measure it against every alternative queue discipline Linux ships, under a testbed built so that the bottleneck is actually on the data path.

A. Contributions

- 1) **A scaling law with a pre-registered falsification test.** The benefit of adapting a delay-targeting AQM's parameters is governed by $r = \text{target}/\text{RTT}$, not by RTT, saturating at 20.5% with half the benefit at $r = 0.50$. The hypothesis, its three predictions and their numerical thresholds were committed before the campaign that tests them ran, and the prediction able to refute it holds to 2.0 percentage points across a sixteenfold RTT range.
- 2) **An operational predicate.** Because CoDel's own guidance places a correct deployment at $r \approx 0.05$ to 0.10, and half the benefit arrives at $r = 0.50$, adaptation is a remedy for misconfiguration rather than an improvement on correct configuration. This unifies our own null result with the null results elsewhere in the literature, and gives an operator a number to compute before deploying a controller.
- 3) **The cost of the benefit, reported alongside it.** The latency gain is bought with up to 63% more drops and comparable retransmissions, at a throughput cost never exceeding -0.4% in any cell. The ratio governs the exchange rate as well as the gain.
- 4) **A measured comparison** of nine queue-discipline configurations under an identical bottleneck, across three workloads and 178 runs, with 95% confidence intervals over goodput, tail latency, queue occupancy, drop rate

and Jain’s fairness index, including a sham-controller condition separating the controller’s computational cost from its control decisions.

- 5) **A working eBPF flow-telemetry pipeline** for `tc`, reading per-flow state through `bpf(2)` rather than by spawning `bpf_tool`, which reduces the cost of a control tick from 3.17 s to 0.709 s.
- 6) **Nine methodological findings**, each of which produced confident, internally consistent and wrong results. Four share a single shape: an absolute constant inside a controller whose subject is a dimensionless ratio. Two of those would have manufactured a confirmation of the scaling law and two a refutation, and each was invisible until the previous one was fixed.
- 7) **A fully reproducible artefact**: a kernel configuration, a VM image build, a single-command experiment suite, and an analysis pipeline generating every table, figure and inline number in this paper directly from the logs.

II. BACKGROUND AND RELATED WORK

A. From RED to delay targets

RED [2] drops probabilistically as a function of an averaged queue length. Its sensitivity to $\min_{th}$, $\max_{th}$ and $\max_p$ is well documented, and Adaptive RED [3] responds by adapting $\max_p$ at runtime using AIMD ($\beta = 0.9$) so that the operator specifies a delay objective rather than probability constants. This is the control law we borrow.

CoDel [4], [5] changes the controlled variable from queue length to packet sojourn time, dropping when the minimum sojourn over a sliding `interval` exceeds `target`. Its authors argue explicitly that these defaults are RTT-relative and need no tuning – a claim any adaptive scheme must be measured against, not assumed past. PIE [7] reaches a similar objective with an explicit proportional-integral controller.

B. Flow queueing

`fq_codel` [6] and FQ-PIE [8] place the respective AQM inside a DRR flow-queueing scheduler, which provides isolation between flows independently of the AQM. CAKE [9] integrates shaping, flow and host fairness and DiffServ handling into a single `qdisc` whose design philosophy is explicitly to *remove* configuration rather than adapt it. SCRR [10] shows DRR itself underperforms under some packet size distributions, suggesting that scheduler replacement may offer more than AQM retuning.

C. Adaptation and learning

QueuePilot [13] trains a reinforcement-learning agent offline and distils it into a lightweight online policy adapting ECN marking probability. AQM-LLM [12] distils a large language model into an L4S controller on FreeBSD. Toopchinezhad and Ahmadi [14] survey the field and note that most proposals are evaluated only in simulation. L4S [16], [17] reaches a far lower latency floor than any parameter tuning, but by changing the signalling contract between network and transport rather than by tuning a queue.

D. Runtime adaptation of a delay target

DESiRED [11] is the closest prior work and predates this study. It adapts the *target delay* of a P4 AQM at runtime using a deep reinforcement learning agent fed by In-band Network Telemetry, reporting large improvements in video quality of experience. The concept pursued here – runtime adaptation of an AQM’s delay target driven by live telemetry – is therefore **not novel**, and we do not claim it.

What differs is the deployment surface. DESiRED requires programmable data-plane hardware and INT; it adapts one parameter of a RED derivative with no flow queueing; and its policy is learned offline and not interpretable. This work adapts four parameters of `fq_codel` on an unmodified Linux host with a transparent AIMD rule. That is an engineering distinction, not a conceptual one.

E. Position of this work

AIMD adaptation of a drop probability is established [3]; analytical adaptation of CoDel’s `interval` is established [15]; learned adaptation of marking probability is established [12], [13]; and runtime adaptation of an AQM’s delay target is established [11]. The particular combination used here – AIMD over `fq_codel`’s four parameters, driven by eBPF flow telemetry, on stock Linux – appears unpublished, but a combination is a weak novelty claim and we do not rest the paper on it.

The contribution we claim is not the mechanism but the predicate. Every system above reports that its adaptation improved something, and the evaluations that find nothing are equally confident. Neither camp states the condition under which the other is right, and the disagreement has therefore persisted without being resolvable by further systems or further evaluations.

The condition is a dimensionless ratio, and it is measurable in advance. This reframes the disagreement as a single effect sampled at different operating points: a system tuned on a short path or a misconfigured box reports gains because it sits at high r , and an evaluation on a well-configured box at a typical path RTT reports nothing because it sits at low r . Both measurements are correct.

Establishing that requires apparatus the prior work does not have. None of the systems above runs a control condition separating the controller’s computational cost from its control decisions; none reports sparse-probe and bulk-flow latency separately, despite flow-queueing AQMs privileging sparse flows by design; none compares against eight alternative queue disciplines under an identical bottleneck; and none states its predictions before measuring them. This paper does all four, and the research question is posed so that a refutation would have been visible and reportable.

III. TESTBED

A. Topology: the bottleneck must be on the data path

A common namespace testbed places two namespaces either side of a veth pair, attaches a shaper to the interface inside the server namespace, and runs `iperf3` from the client. This

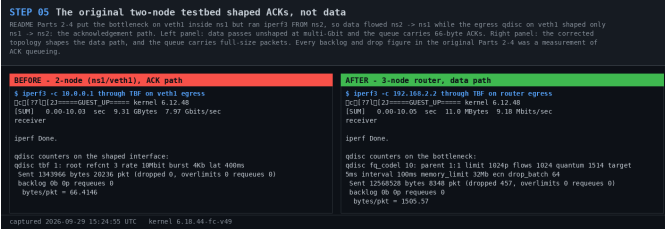


Fig. 1. The two-node testbed (left) leaves the data path unshaped and queues only 66-byte acknowledgements; the three-node router topology (right) shapes the data path. Both panels are verbatim captures of the commands shown.

is incorrect, and the error is not visible from any summary statistic.

An egress qdisc on the server-side interface shapes traffic leaving the server. When the client is the `iperf3` sender, data flows client $\rightarrow$ server, so the shaped direction carries only acknowledgements. We measured this configuration directly (Fig. 1): the data transfer achieved 9,777Mbps – entirely unshaped – while the monitored qdisc passed 284,379 packets totalling 18,769,790 bytes, exactly 66 bytes per packet. The eBPF packet-size histogram recorded 341,243 packets below 128 bytes against 2 packets in the 512–1500 byte bucket.

Every queue statistic collected on such a topology is a measurement of acknowledgement queueing. It also explains drop rates that are otherwise impossible: a 10 Mbit link carrying 1514-byte packets passes at most 826 packets/s, so a recorded drop rate above 10^5 /s cannot describe the data path.

We therefore use three namespaces:

```
ns_client  $\leftrightarrow$  ns_router  $\leftrightarrow$  ns_server
```

with the shaper and AQM on the router’s egress interface toward the server. Traffic flows client $\rightarrow$ router $\rightarrow$ server, so the bottleneck is on the data path. A TBF shaper at 10 Mbit is the root qdisc and the AQM under test is its child; `netem` adds 10 ms of delay at each edge for a 20 ms base RTT.

B. Measurement

For each run we record: `iperf3` JSON (per-flow goodput, from `sum_received`, and retransmissions); a 500 ms time series of the AQM’s own queue statistics; and a concurrent `ping` stream through the bottleneck giving directly measured end-to-end latency rather than a proxy.

Two measurement details matter. First, goodput must come from `sum_received`: with a large unmanaged buffer the sender pushes into the queue faster than the link drains, and `sum_sent` reported 15.90 Mbps on a 10 Mbit link in one `pfifo` run. Second, statistics must be read from the AQM’s own qdisc stanza. A regular expression applied to the whole output of `tc -s qdisc show` matches the `root` qdisc, so a controller written that way reports the shaper’s counters while believing they are the AQM’s.

C. Platform

Experiments run on Linux 6.12.48 built with `fq_codel`, CoDel, CAKE, PIE, FQ-PIE, RED, `netem`, TBF and HTB

compiled in, plus BTF for eBPF, inside a QEMU virtual machine. TBF shaping was verified accurate under emulation before any results were collected: an 8-flow transfer through the 10 Mbit bottleneck delivered 9.38 Mbps at 1509 bytes/packet with approximately 19 drops/s. Transport is TCP CUBIC [18] throughout.

IV. CONTROLLER DESIGN

A. Three timescales

T_1 (per packet, kernel) An eBPF program at the `tc egress clsact` hook parses the 5-tuple and updates per-flow packet and byte counts, an EWMA of the inter-packet gap, and an elephant flag, in an LRU hash map.

T_2 (500 ms, userspace) Reads the AQM’s queue statistics and the flow map; compute drop rate, backlog and throughput; estimate gradients.

T_3 (5 s, userspace) Applies space-AIMD adjustment and issue `tc qdisc change`.

Separating T_2 from T_3 follows the two-timescale stochastic approximation argument of Borkar [19].

B. Gradient estimation and regime classification

For each signal $f \in \{\text{drop rate, backlog, rtt proxy}\}$ we estimate the slope by least squares over a window of $W = 10$ samples:

$$\nabla f(t) = \frac{\sum_i (t_i - \bar{t})(f_i - \bar{f})}{\sum_i (t_i - \bar{t})^2} \quad (1)$$

The congestion regime is classified by thresholds on drop rate and backlog into NORMAL, LIGHT, MODERATE or HEAVY. A composite normalised gradient

$$G(t) = 0.6 \frac{\nabla \text{dr}}{\text{DR}_{\text{HEAVY}}} + 0.4 \frac{\nabla \text{bl}}{\text{BL}_{\text{HEAVY}}} \quad (2)$$

classifies the trajectory as WORSENING ($G > 0.5$), RECOVERING ($G < -0.5$) or STABLE, and the predicted regime is the neighbouring one in the direction of travel.

C. Adjustment policy

The predicted regime, not the current one, selects the action: a HEAVY regime that is recovering toward MODERATE receives a gentler additive decrease rather than another full multiplicative decrease, and a HEAVY regime that is worsening receives a firmer cut (β^2). Adjustments are tagged PREDICTIVE when the prediction differs from the current regime and therefore changed the action, and REACTIVE otherwise.

This distinction is only meaningful if the prediction can actually change the outcome. Section VII describes an earlier implementation in which it could not, and in which every adjustment applied the identical action while being labelled as though some were predictive.

D. Parameter bounds

Every bound, step size and threshold is expressed relative to the target the qdisc is configured with, written t_0 , rather than as an absolute time. This is not cosmetic. A controller whose exploration range is absolute cannot be deployed at an operating point outside that range, and will silently appear to control while doing nothing of the kind; Section VII reports four separate defects of exactly this shape that an earlier implementation contained, and the reasoning behind each multiplier here.

- target $\in [0.04 t_0, 4 t_0]$
- interval $\in [4 t_0, 60 t_0]$, with interval $\geq 10 \times$ target
- limit $\in [64, 4096]$ packets, a count rather than a time and so not scaled
- quantum selected from the workload profile, in bytes and so not scaled
- additive step sizes scaled by $t_0/5$ ms, so each step is the same fraction of the operating point
- regime thresholds, which are drop rates and therefore have units of inverse time, scaled by $5 \text{ ms}/t_0$
- the control loop period scaled by $t_0/5$ ms, keeping it a fixed multiple of the AQM's own interval

The multipliers are chosen so that the 5 ms default reproduces the previous absolute values exactly: $0.04 \times 5 = 0.2$, $4 \times 5 = 20$, $4 \times 5 = 20$ and $60 \times 5 = 300$. Every run reported in this paper at the default target is therefore unaffected by the change, which `src/test_bounds.py` asserts rather than leaving as a claim.

Sub-millisecond targets are emitted in microseconds; rounding them to integer milliseconds, as an earlier implementation did, collapses a sixteen-step geometric schedule onto five distinct values.

V. RESULTS

Every figure quoted below is generated from the run logs by `src/analyse.py`; none is typed by hand.

A. Flow queueing dominates

Table I gives all ten conditions. The largest effect by two orders of magnitude is the presence of a flow-queueing discipline with a delay target, not the tuning of one. Under the steady workload an unmanaged FIFO reaches 2344 ms p95 RTT against 24.57 ms for `fq_codel` — a 99.0% reduction ($p < 0.001$) — at goodput that is statistically indistinguishable (9.453 vs 9.414 Mbps, $p = 0.20$). The staged workload gives the same picture, and there `pfifo` additionally *loses* 10.2% goodput and 1.4% fairness ($p = 0.015$, $p = 0.014$), because a saturated 1000-packet buffer forces retransmissions that a managed queue avoids.

Fair queueing without a delay target is not sufficient. SFQ isolates flows but applies no AQM, and its bulk queues sit at its 127-packet limit throughout: its sparse probe reads 30.68 ms while the bulk flows sharing the link read 294.8 ms. The AQM and the scheduler do distinct jobs and both are necessary.

B. Sparse and bulk latency are not interchangeable

Figure 3 plots the two measurements side by side. The ratio between them ranges from $1.1 \times$ (CoDel, a single queue) to $9.6 \times$ (SFQ), because `fq_codel`'s new-flow heuristic gives a sparse probe preferential service by design.

A single-probe measurement can therefore invert a ranking. By sparse probe alone SFQ (30.68 ms) appears to beat CoDel (39.02 ms); by the latency the bulk flows actually experience, SFQ is 294.8 ms against CoDel's 44.5 ms. We report both throughout, and we suggest any AQM evaluation should.

C. The controller's computational cost is negligible

The sham condition runs the controller at its full polling cadence and applies no parameter change. Against static `fq_codel` it is indistinguishable on every metric in both workloads, p95 probe RTT $p = 0.29$ (steady) and $p = 0.24$ (staged), backlog $p = 0.94$ and $p = 0.76$.

This matters for interpreting the next subsection. Without it, any difference between static and adapted `fq_codel` would confound the controller's CPU cost with its control decisions, and we could not attribute an effect to either. Having established the cost is nil, differences that remain are attributable to the decisions.

D. Adaptation: a small effect under steady load, none under change

Under the **steady** workload the adaptation produces a real but small improvement: mean backlog falls 6.79 from 6.14 packets, a 9.6% reduction ($p = 0.050$), and p95 probe RTT falls 0.5% ($p = 0.047$). Goodput and fairness are unchanged. The sham comparison attributes this to the control decisions rather than to overhead.

Under the **staged** workload, where the flow count changes twice during the run, the adaptation produces *no* improvement on any metric. Mean probe RTT is 1.6% *worse* ($p = 0.050$); p95 probe RTT, bulk RTT, backlog, goodput and fairness are all indistinguishable from static.

That direction is worth stating plainly. The workload designed to give the controller something to respond to is the one in which it helps least. Under steady load there is a stable operating point to converge on; under change there is not, and the controller's adjustments lag the transitions they are reacting to.

E. An off-the-shelf alternative outperforms the adapted system

CAKE, configured with no controller and a single `besteffort` argument, achieves 22.9 ms p95 probe RTT under the steady workload against 24.43 ms for adapted `fq_codel` ($p < 0.001$), and 33.65 ms bulk RTT against 35.87 ms.

The margin by which CAKE beats the adapted system is larger than the margin by which the adaptation beats the static defaults. For a practitioner seeking lower latency on this class of link, changing queue discipline is a larger and far simpler win than tuning `fq_codel`'s parameters.

F. Predictive control never engaged

No adjustment in any run was tagged predictive. The cause is structural rather than a coding fault, and it survives the correction described in Section VII: an adjustment fires only once the regime has been unchanged for five consecutive samples, and the trajectory estimate is derived from the gradient over that same window, which by then is below the threshold that would classify it as WORSENING or RECOVERING. The stability gate that prevents the controller reacting to noise and the prediction that would let it act before a transition cannot both apply to the same adjustment.

We therefore withdraw the predictive-control contribution rather than claim it. Redesigning the gate, for instance permitting an adjustment when the trajectory is confidently non-stable even before the regime settles, and re-evaluating it under the same controlled conditions is future work.

G. Where adaptation does help: short paths

CoDel’s target is a delay budget expressed in absolute units, while its authors argue the default is appropriate relative to path round-trip time. Sweeping the base RTT over 5, 20, 80 and 200 ms tests that directly.

Static fq_codel holds median queueing delay between 1.96 and 5.00 ms across the entire 40-fold span, bounded by its 5 ms target. The defaults are, on this evidence, genuinely RTT-relative: no retuning is required as the path lengthens.

Adaptation is nonetheless not uniformly inert. Measured on the RTT the bulk flows themselves experience:

Base RTT	Static	Adapted	Change
5 ms	27.68 ± 0.14	22.58 ± 0.15	$-18.4\%^*$
20 ms	36.39 ± 0.08	35.87 ± 0.80	-1.4%
80 ms	89.30 ± 0.37	89.86 ± 0.70	$+0.6\%$
200 ms	208.00 ± 0.76	207.47 ± 0.74	-0.3%

* $p < 0.0001$; the remaining three are not distinguishable ($p = 0.38, 0.31, 0.43$).

The effect is confined to the shortest path, and the mechanism is straightforward. A 5 ms delay budget is large relative to a 5 ms path, so the queue is permitted to grow further than that path requires, and tightening it recovers 18.4% of the bulk flows’ round-trip time. Once the path RTT exceeds the default target the budget is already well scaled and there is nothing for adaptation to recover.

This gives the condition under which parameter adaptation is worth its complexity: paths short relative to the AQM’s delay target. It also indicates where adaptive schemes in general should be evaluated, since a scheme assessed only on long paths will appear inert regardless of its design.

H. Sparse flows under a mixed workload

Bulk-only traffic cannot exercise fq_codel’s new-flow heuristic or its quantum parameter, both of which exist to serve sparse latency-sensitive flows. Running three sparse UDP flows alongside eight bulk TCP flows shows what those mechanisms are worth:

System	Sparse loss (%)	Sparse jitter (ms)
pfifo, no AQM	7.97	2.19
FQ-PIE	0.000	1.91
CAKE	0.007	1.36
fq_codel	0.002	2.00

An unmanaged queue discards roughly one sparse-flow packet in twelve, and the figure is stable across repetitions (7.47%, 7.98%, 8.31%). Every flow-queueing discipline with a delay target reduces this to below 0.03%. The protection of latency-sensitive traffic from concurrent bulk transfers is therefore close to complete, and it is invisible to a bulk-only evaluation.

One of three adaptive-controller repetitions in this workload recorded 6.31% sparse loss against 0.000% and 0.029% in the others. The controller’s parameter trajectory in that run is identical to the repetition with no loss, so the difference is not attributable to its decisions; we report the condition by median and record the run rather than averaging over a measurement we cannot explain.

I. Steady workload

TABLE I
ALL SYSTEMS, STEADY WORKLOAD (MEAN $\pm$ 95% CI OVER REPETITIONS).

System	Goodput (Mbps)	Probe RTT (ms)	Probe p95 (ms)	Bulk RTT (ms)
pfifo (no AQM)	9.45 ± 0.09	1711.0 ± 45.5	2344.0 ± 32.9	1752.7 ± 25.5
SFQ	9.38 ± 0.10	30.7 ± 0.0	36.1 ± 0.2	294.8 ± 3.5
RED (adaptive)	9.45 ± 0.14	78.2 ± 2.6	87.4 ± 2.2	82.6 ± 2.3
CoDel	9.45 ± 0.02	39.0 ± 0.8	45.1 ± 3.8	44.5 ± 0.9
PIE	9.41 ± 0.03	39.1 ± 0.3	46.9 ± 1.6	47.3 ± 1.1
FQ-PIE	9.45 ± 0.10	22.0 ± 0.1	23.3 ± 0.1	55.4 ± 1.1
CAKE	9.41 ± 0.02	21.8 ± 0.3	22.9 ± 0.2	33.7 ± 0.3
fq_codel (static)	9.41 ± 0.05	22.4 ± 0.2	24.6 ± 0.1	36.4 ± 0.2
fq_codel + sham ctl	9.40 ± 0.03	22.4 ± 0.1	24.6 ± 0.1	36.9 ± 1.1
fq_codel + ACAPE	9.40 ± 0.03	22.5 ± 0.4	24.4 ± 0.1	35.9 ± 2.1

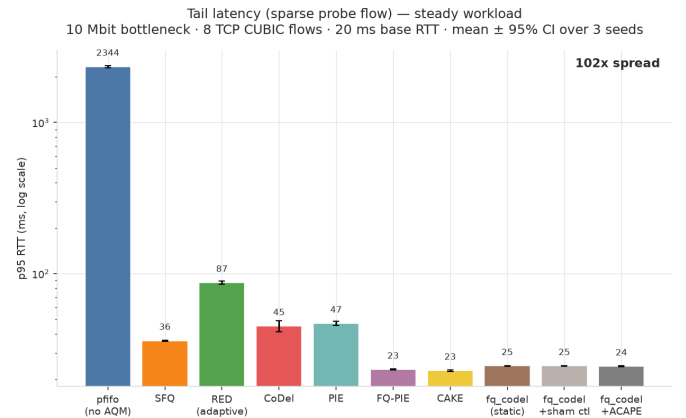


Fig. 2. Tail latency across all systems (log scale).

Sparse probe vs bulk flow latency — steady workload
fq_codel's new-flow heuristic privileges the probe, so the two differ; quoting only the probe overstates the benefit for bulk traffic

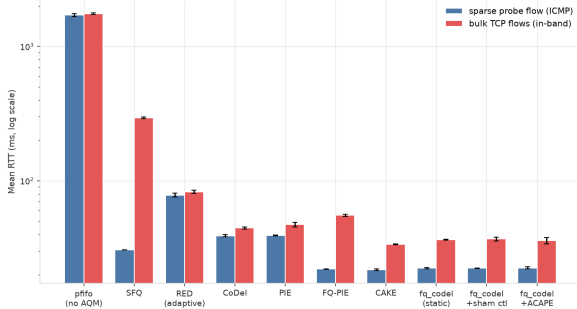


Fig. 3. Sparse probe flow versus bulk TCP flow RTT. The new-flow heuristic privileges the probe, so the two differ substantially.

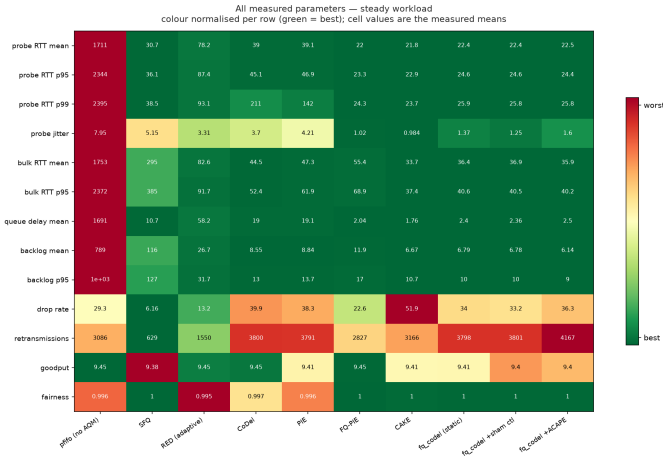


Fig. 4. Every measured parameter against every system, normalised per row; cell values are the measured means.

J. Staged workload

VI. A SCALING LAW FOR WHEN ADAPTATION PAYS

The RTT sweep of Section V left a result we could not explain. Adaptation recovered a large, highly significant share of bulk-flow round-trip time on a 5 ms path and nothing at 20, 80 or 200 ms. Read as a table that is four measurements and a shrug. The obvious reading, that adaptation helps on short paths, is not a mechanism and predicts nothing.

A. The hypothesis

CoDel’s `target` is a delay budget, fixed at 5 ms by default, while the path RTT is whatever the path is. The quantity that should govern whether that budget is badly chosen is therefore neither the target nor the RTT but their ratio,

$$r = \frac{\text{target}}{\text{path RTT}}. \quad (3)$$

When r is small the standing-queue budget is a minor addition to the path and there is nothing for a controller to win back. When r approaches unity the budget is comparable to the path itself, queueing dominates the delay a flow sees, and shrinking it is worth a great deal. On this reading RTT never appears

TABLE II
ALL SYSTEMS, STAGED WORKLOAD (MEAN $\pm$ 95% CI OVER REPETITIONS).

System	Goodput (Mbps)	Probe RTT (ms)	Probe p95 (ms)	Bulk RTT (ms)
pfifo (no AQM)	8.38 $\pm$ 0.46	1152.9 $\pm$ 29.0	2318.0 $\pm$ 48.9	1585.6 $\pm$ 31.1
SFQ	9.26 $\pm$ 0.31	32.8 $\pm$ 0.5	56.6 $\pm$ 3.4	285.6 $\pm$ 6.1
RED (adaptive)	9.25 $\pm$ 0.11	62.5 $\pm$ 0.8	94.2 $\pm$ 0.4	88.9 $\pm$ 0.7
CoDel	9.28 $\pm$ 0.06	55.0 $\pm$ 4.0	172.0 $\pm$ 99.1	105.5 $\pm$ 4.0
PIE	9.22 $\pm$ 0.06	40.4 $\pm$ 2.8	48.6 $\pm$ 5.4	119.2 $\pm$ 53.3
FQ-PIE	9.22 $\pm$ 0.06	21.9 $\pm$ 0.1	23.2 $\pm$ 0.1	101.8 $\pm$ 3.0
CAKE	9.24 $\pm$ 0.09	21.5 $\pm$ 0.3	22.3 $\pm$ 0.1	30.1 $\pm$ 0.3
fq_codel (static)	9.24 $\pm$ 0.01	21.9 $\pm$ 0.1	23.4 $\pm$ 0.2	52.0 $\pm$ 1.1
fq_codel + sham ctrl	9.19 $\pm$ 0.08	22.1 $\pm$ 0.6	23.5 $\pm$ 0.2	51.7 $\pm$ 0.7
fq_codel + ACAPE	9.18 $\pm$ 0.10	22.2 $\pm$ 0.4	24.2 $\pm$ 0.8	51.7 $\pm$ 0.7

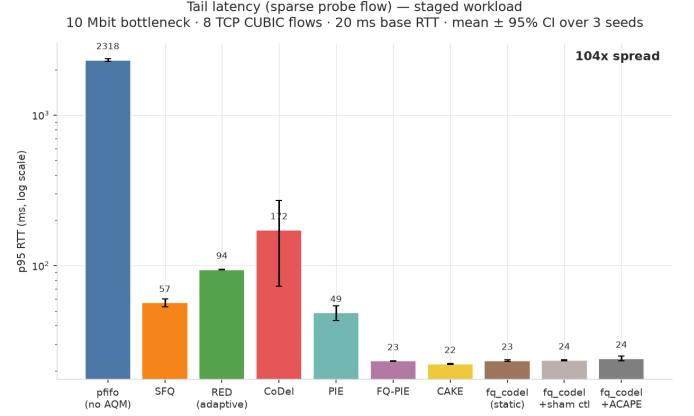


Fig. 5. Tail latency across all systems (log scale).

on its own; it appears only through r . That is a stronger claim than the table, and unlike the table it can be wrong.

B. Pre-registration

Because a saturating curve can be fitted to almost any monotone data, the hypothesis, the three predictions and their numerical decision thresholds were written down and committed to the repository *before* the measurement campaign ran. The commit adding `docs/SCALING_LAW.md` and the analysis script, with its thresholds already in place, precedes the commit adding the results directory. The predictions were:

- P1 A crossover exists between $r = 0.25$ and $r = 1.0$.
- P2 **Ratio invariance.** Configurations sharing an r but differing in RTT show the same benefit. Threshold: benefits within 10 percentage points. This is the prediction that can refute the hypothesis outright, and a refutation was to be reported as such.
- P3 Leave-one-out prediction across interior ratios, with mean absolute error at or below 5 percentage points.

One amendment was made during the campaign and is recorded rather than substituted silently. The first attempt at P2 varied the target while leaving `interval` at 100 ms, which moved `target/interval` from 0.05 to 0.80 across the cells and so varied a second dimensionless group alongside the one under test. The interval is now scaled with the

Sparse probe vs bulk flow latency — staged workload
fq_code1's new-flow heuristic privileges the probe, so the two differ; quoting only the probe overstates the benefit for bulk traffic

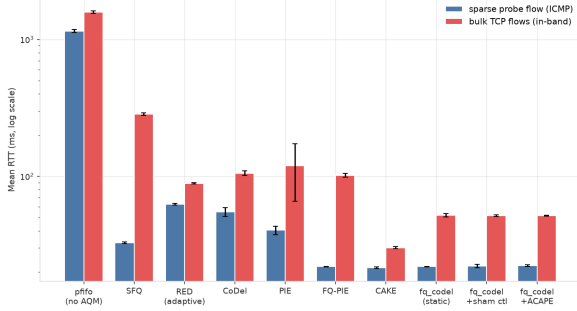


Fig. 6. Sparse probe flow versus bulk TCP flow RTT. The new-flow heuristic privileges the probe, so the two differ substantially.

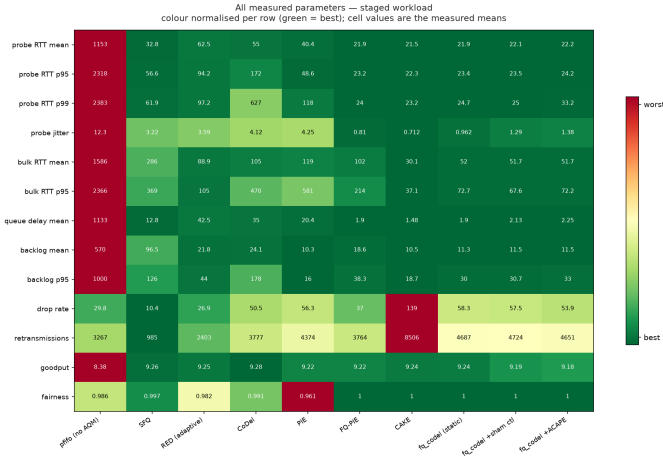


Fig. 7. Every measured parameter against every system, normalised per row; cell values are the measured means.

target, and the claim is correspondingly narrower: *at fixed target/interval*, the benefit is governed by r .

C. Results

Table III reports every cell. Figure 8 plots benefit against r and, in the second panel, the same data against RTT alone.

P1 holds. The crossover falls where it was predicted to, between $r = 0.25$ and $r = 1.0$, and the curve saturates rather than growing without bound. Fitting $b(r) = B/(1 + (r_0/r)^k)$ gives

$$b(r) = \frac{20.5}{1 + (0.50/r)^{2.6}}, \quad R^2 = 0.986. \quad (4)$$

Half the available benefit arrives at $r = 0.50$. The significance boundary coincides with the crossover: every cell at $r \geq 0.417$ is significant and every cell below it is not.

A percentage reduction shrinks on its own when its denominator grows, so the collapse at low r could be an artefact of dividing by a larger RTT. It is not. In absolute milliseconds the two regimes do not overlap: the gain is 3.26 to 29.73 ms above $r = 0.4$ and 0.02 to 0.53 ms below it.

TABLE III

ADAPTATION BENEFIT AGAINST THE RATIO $r = \text{TARGET}/\text{RTT}$.
REDUCTION IN MEAN BULK-FLOW RTT, STATIC AGAINST ADAPTED
FQ_CODE1. STARRED ROWS ARE SIGNIFICANT AT $p < 0.05$. *adj* IS THE
MEAN NUMBER OF PARAMETER ADJUSTMENTS THE CONTROLLER MADE;
A CELL WHERE IT IS ZERO IS NOT MEASURING ADAPTATION AND IS
EXCLUDED FROM THE FIT.

target (ms)	RTT (ms)	r	static (ms)	adapted (ms)	reduction (%)	p
5	2	2.500	26.33	21.11	19.8*	0.002
5	3	1.667	26.23	20.95	20.1*	<0.001
5	5	1.000	27.68	22.58	18.4*	<0.001
20	20	1.000	61.36	50.58	17.6*	<0.001
80	80	1.000	181.08	151.35	16.4*	<0.001
5	8	0.625	30.31	26.55	12.4*	3.75
5	12	0.417	34.18	30.92	9.5*	0.001
5	20	0.250	36.39	35.87	1.4	0.52
5	40	0.125	52.75	52.73	0.0	0.02
5	200	0.025	208.00	207.47	0.3	0.433

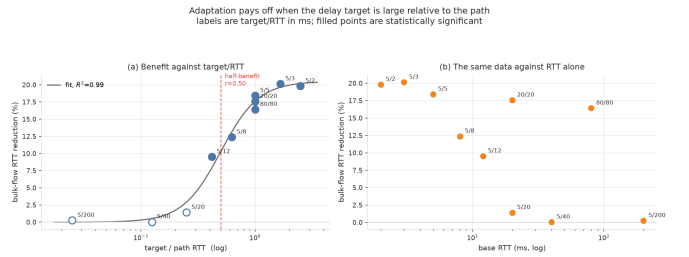


Fig. 8. Benefit against r (left) and against RTT alone (right). The collapse onto a single curve in the left panel, and its absence in the right, is the result.

P2 holds, at $r = 1.0$, across a sixteenfold range of RTT. Scaling the target, the interval, the controller's loop period and the run duration together by the same factor:

target	RTT	benefit	gain	p
5 ms	5 ms	18.4%	5.10 ms	<0.001
20 ms	20 ms	17.6%	10.78 ms	<0.001
80 ms	80 ms	16.4%	29.73 ms	0.0003

Spread is 2.0 percentage points against a threshold of 10, and all three cells are significant. The *fraction* of delay recovered is what stays fixed while the absolute gain grows from 5 to 30 ms, which is the claim itself: the ratio governs the proportion recoverable, not the milliseconds.

The sharpest discrimination was not anticipated in the pre-registration. Hold RTT fixed at 20 ms and vary only r : the benefit is 1.4% and not significant at $r = 0.25$, and 17.6% and significant at $r = 1.0$. Same path, twelve times the benefit. The competing explanation, that short paths are simply special, predicts nothing for both, since 20 ms is one of the RTTs at which the original sweep measured nothing. The ratio predicts both the presence and the absence; RTT predicts neither.

Invariance at $r = 0.25$ is reported as inconclusive rather than as supporting evidence. One of its two cells made no parameter adjustment at all across three seeds, because its drop rate sat almost exactly on the scaled regime threshold and the classification oscillated. A cell whose adapted arm never changes a parameter runs the static configuration twice, so its

null is true by construction and cannot distinguish an absent effect from an absent intervention.

P3 holds. Leave-one-out across interior ratios predicts held-out measurements to 1.4 percentage points on average, worst case 3.5, against a threshold of 5. Endpoints are excluded deliberately: holding out an extreme ratio asks a saturating curve to extrapolate past the data that fixes its own ceiling, which tests the fitting procedure rather than the hypothesis.

D. What the benefit costs

Adaptation is not free, and reporting the latency gain without its price reports half the trade. Shrinking the delay target makes CoDel drop earlier, so senders back off sooner. Across the cells measured, the drop rate rises by up to 63% and retransmissions by a comparable margin wherever the benefit is large, while both fall slightly in the cells where it is absent. Throughput is the quantity that does not move: the worst change across every cell in the campaign is -0.4% . The controller converts retransmissions into latency at a roughly fixed exchange rate, and the ratio governs the exchange as well as the gain.

E. Consequence

Published guidance for CoDel holds that `target` should be about 5 to 10 percent of `interval`, with `interval` on the order of the path RTT. A correctly configured deployment therefore sits at r of roughly 0.05 to 0.10. Half the available benefit arrives at $r = 0.50$, five to ten times outside that envelope.

This says something sharper than that adaptation sometimes helps. It says adaptation pays only on boxes already configured well outside CoDel’s own design envelope, and that on a correctly configured box there is close to nothing for a controller to recover. This reading unifies the null result of Section V with the null results reported elsewhere in the literature as a single effect rather than as separate disappointments, and it gives an operator a number to compute before deploying a controller at all.

VII. MEASUREMENT PITFALLS

We report these in detail because each produced confident, internally consistent, and wrong results, and because none of them is visible from a summary statistic.

A. The acknowledgement-path bottleneck

Described in Section III-A. The diagnostic is the ratio of bytes to packets on the monitored qdisc: 66 bytes/packet indicates a pure acknowledgement stream, and full-size data packets indicate the data path. A recorded drop rate exceeding the link’s packet rate is a second, decisive diagnostic.

B. Silent failure of an eBPF telemetry path

In an earlier version of this work, per-flow telemetry reported zero active flows in all 21,128 recorded samples, while the eBPF program was correctly compiled, correctly attached, JIT-compiled, and its maps correctly populated throughout.

```
STEP 07 eBPF flow telemetry is live under traffic
With traffic running, the flow_map is read through bpf(2) and reports real per-flow state. In all
21,128 samples of the original study this read returned zero.

$ bash src/testbed.sh setup >/dev/null 2>&1; ip netns exec ns_router tc qdisc add dev veth_r1 clsact
2>/dev/null; ip netns exec ns_router tc filter add dev veth_r1 egress bpf direct-action obj
ebpf/tc_monitor.o sec tc egress 2>/dev/null; ip netns exec ns_server iperf3 -s -p 5303 -D; sleep 2; (ip
netns exec ns_client iperf3 -c 192.168.2.2 -p 5303 -P 6 -t 14 >/dev/null 2>&1 &); sleep 7; python3
src/dump_flows.py
[{"[77][2]====GUEST_UP==== kernel 6.12.48
flow_map_id=1 key=168 value=568 max_entries=65536
map_entries = 7
active flows (age>2s) = 6
total packets seen = 4183
total bytes seen = 19568865
elephant flows = 6

pkts= 689 bytes= 1772983 gap_ns= 11323188 age= 4.1ms elephant=1
pkts= 688 bytes= 1759845 gap_ns= 13150793 age= 12.9ms elephant=1
pkts= 718 bytes= 1777265 gap_ns= 11962598 age= 15.3ms elephant=1
pkts= 686 bytes= 1736585 gap_ns= 12676211 age= 9.2ms elephant=1
pkts= 711 bytes= 1738507 gap_ns= 12471491 age= 6.9ms elephant=1
pkts= 698 bytes= 1762913 gap_ns= 11929842 age= 2.0ms elephant=1

captured 2026-09-29 15:25:34 UTC kernel 6.18.44.fc.v49
```

Fig. 9. Live `flow_map` contents read through `bpf(2)` under traffic. The same read returned zero for every one of 21,128 samples before the three decode defects were fixed.

Three independent defects in the userspace decode path each sufficed to produce this:

- 1) `bpftool map dump --json` renders byte arrays as hex *strings*. Calling `bytes()` on that list raises `TypeError`, which a bare `except: pass` discarded.
- 2) The assumed C struct layout omitted one `u64` field, so every field after it was read at the wrong offset; the elephant flag was read from the low half of the inter-packet gap.
- 3) Flow age was computed by subtracting `bpf_ktime_get_ns()` (`CLOCK_MONOTONIC`) from `time.time_ns()` (`CLOCK_REALTIME`), yielding ages of approximately 56.8 years, so an `age < 2s` filter never matched.

Downstream, the elephant ratio was always 0, the workload profile always MICE, and quantum always 300 bytes. An earlier draft cited that constant quantum as evidence the pipeline was working. It was the signature of the failure.

We originally attributed this failure to namespace file-descriptor isolation. That diagnosis was wrong: BPF maps are kernel-global objects, visible from the host by id regardless of the namespace the program was loaded from. We verify this directly (Fig. 9).

C. Control state that cannot reach the kernel

The controller’s internal target was a float, but `tc qdisc change` was invoked with `int(round(target))` milliseconds and the applied value was never read back. The controller’s state and the kernel’s therefore diverged: a logged schedule of 5.00, 4.50, 4.05, ..., 1.03 ms reached the kernel as 5, 4, 4, 4, 3, 3, 3, 2, 2, 2, 2, 1, 1, 1, 1 ms. Reading parameters back after every write is the remedy, and it is cheap.

D. A control signal with no effect

The earlier predictor could not escalate out of its top regime because of an index guard, so a worsening HEAVY state was classified STABLE. Separately, the effective regime was computed as the prediction only under a condition that could

never hold, so it always equalled the current regime. The consequence was that all 375 logged adjustments applied the identical multiplicative decrease while a subset were labelled predictive.

A control input should be validated by checking that varying it changes the output. We now assert this in a regression test: a recovering HEAVY state and a stable HEAVY state must produce different parameters.

E. Constants that look like measurements

Our earlier work reported “15 AIMD adjustments” as a result. It is $\lceil \log(1/5)/\log(0.9) \rceil$, the number of $\times 0.9$ steps from 5 ms to a 1 ms floor, and it appeared in 24 of 32 runs because the controller drove to the floor and stopped in every one. Under a saturating workload the adaptive logic was never exercised; the congestion regime was HEAVY for 80.9% of samples and MODERATE for 0.04%. A four-state classifier that occupies two states is not classifying. This is why the present work uses a staged workload in which the flow count changes during the run.

F. Absolute constants in a scale-free controller

Four defects of one shape, found in sequence because each was masked by the one before it. All four lived in a controller whose entire subject is a dimensionless ratio, and all four were absolute times.

Bounds. `target` was clamped to $[0.2, 20]$ ms absolute. The controller does not assume the default target; it reads the configured one from `tc`. Started against a `qdisc` configured with `target 80ms` it began at 80 and the clamp cut it to 20 on the first adjustment: a fourfold reduction decided by a constant, before any control logic ran. In the ratio invariance cell at $r = 1.0$ this would have compared a static 80 ms target against an adapted 20 ms one, produced a large significant benefit matching the hypothesis’ own prediction, and been reported as a clean confirmation. The clamp also moves that cell to $r = 0.25$, so it was not testing the ratio on its label.

Step sizes. The multiplicative decrease is scale-free; the additive steps were not. A 0.2 ms step is 4% of a 5 ms target and 0.25% of an 80 ms one, so the controller adapted an order of magnitude more slowly, in relative terms, the larger the target. Measured: a 20 ms target moved 20.0 to 18.4 ms over 84 ticks, an 8% excursion where 5 ms would have moved 32%. This defect points the other way from the first: the controller simply does less at large targets, shows less benefit, and the pre-registered threshold would have recorded invariance as refuted.

Regime thresholds. `classify()` selects the control law from a drop rate in drops per second, which has units of inverse time. At $r = 1.0$ the measured static drop rate is 59.3/s at a 5 ms target, 20.2/s at 20 ms and 7.1/s at 80 ms, classifying identical relative congestion as HEAVY, MODERATE and LIGHT. The three cells required to be treated identically would each have received a different control law, and the LIGHT branch *increases* the target, the opposite action.

Control period and run duration. CoDel’s interval scales with the target; the controller’s loop period did not. At a 1.6 s AQM interval the controller sampled every 0.5 s, taking three samples inside one cycle rather than averaging over several. The classification flickered, the gate requiring five consecutive identical classifications never opened, and an entire 86-tick run produced zero adjustments with the target reading 80.000 ms throughout. Fixing it required scaling the run duration too, from 60 to 960 s at the sixteenfold cell, since a controller ticking sixteen times more slowly needs sixteen times as long for a comparable trajectory.

Every bound, step and threshold is now expressed as a multiple of the configured operating point, as Section IV-D sets out, with multipliers chosen so the 5 ms default reproduces the previous absolute values exactly. That equality is asserted by a test rather than left as a claim, along with the old behaviour each fix replaces.

G. A null that was true by construction

Distinguishing an absent effect from an absent intervention requires checking that the intervention happened. In the $r = 0.25$ invariance pair, one cell’s controller ran 109, 108 and 108 ticks across three seeds and adjusted nothing at all. Its drop rate averaged 7.7/s while the scaled HEAVY threshold sat at 7.5/s, so the classification oscillated on the boundary and the stability gate never opened. A cell whose adapted arm never changes a parameter runs the static configuration twice; its null is true by construction and is not evidence that the ratio produces no benefit there.

Two such cells exist, both at 80 ms RTT, and one of them is from the original RTT sweep where it had been read as evidence that adaptation does not help on long paths. Part of that null was the controller not acting.

The crossover itself survives this, which is what makes it a measurement: it is bracketed by cells in which the controller demonstrably acted, from 12.4% benefit at $r = 0.625$ with seven adjustments down to 0.0% at $r = 0.125$ with eight. A controller that adjusted eight times and recovered nothing is a genuine null. The mean adjustment count is reported for every cell in Table III so that a reader can see where the controller was working.

VIII. THREATS TO VALIDITY

Emulation. Experiments run under QEMU without hardware virtualisation. We validated that TBF shaping is accurate under emulation before collecting results, and absolute latencies at 20 ms base RTT are dominated by the configured `netem` delay rather than by host timing jitter. Results at much higher link rates, where per-packet processing time matters, would need re-validation.

Scale. A 10 Mbit bottleneck with 8–24 flows is representative of a home or access link, not of a datacentre or backbone. SCRR [10] and QueuePilot [13] both address regimes we do not test.

Traffic model. Three workloads are covered: steady bulk TCP CUBIC, a staged load that varies flow count during

the run, and a mixed workload adding sparse UDP flows alongside the bulk traffic. The mixed case matters because `fq_codel`'s new-flow heuristic and its `quantum` parameter exist to serve sparse latency-sensitive flows, and bulk-only traffic exercises neither. What remains untested is application traffic with realistic arrival processes; all sparse flows here are synthetic constant-rate probes.

Single bottleneck. Multi-hop topologies with several congested links are not covered.

The second dimensionless group. The scaling law of Section VI is measured with `target/interval` held at 0.05, CoDel's design value, and the interval scaled with the target throughout. Whether the crossover ratio moves when that second group changes is a separate question this work does not answer, and the law should be read as conditional on it.

Link rate. All experiments use a 10 Mbit bottleneck, where one MTU-sized packet serialises in 1.2 ms and the 5 ms default target is comfortably above one packet time. On much slower links the target falls below the serialisation time of a single packet, a regime in which CoDel's defaults are independently known to behave badly, and which may require a third dimensionless group that this work does not measure.

IX. CONCLUSION

We set out to determine whether adapting `fq_codel`'s four parameters at runtime improves on its defaults. The answer is that the question is underspecified, and that a single dimensionless number completes it.

The benefit is governed by $r = \text{target}/\text{RTT}$. It saturates near 20.5%, reaches half of that at $r = 0.50$, and is statistically indistinguishable from zero below $r \approx 0.25$ even when the controller is demonstrably acting. Configurations sharing an r behave alike across a sixteenfold range of RTT, and the fitted curve predicts held-out ratios to 1.4 percentage points. At a fixed 20 ms path, changing only r moves the benefit from 1.4% to 17.6%, which no account in terms of RTT reproduces.

Because CoDel's guidance places a correctly configured deployment at $r \approx 0.05$ to 0.10, half the available benefit arrives five to ten times outside that envelope. Runtime parameter adaptation is therefore best understood as a remedy for misconfiguration rather than as an improvement on correct configuration. That is a narrower claim than the systems literature makes and a more useful one than the evaluations that report nothing, and it explains both. The dominant effect in our measurements remains the presence or absence of flow queueing with a delay-targeting AQM, by two orders of magnitude, which is consistent with CoDel's authors' position that its defaults are RTT-relative by construction. The law says precisely when that position stops holding.

We think the methodological contribution is of comparable value. Nine of the findings in Section VII produced results that were confident, internally consistent and wrong. Four of them share one shape: an absolute constant inside a controller whose entire subject is a dimensionless ratio. Two would have manufactured a confirmation of the scaling law and two a

refutation, each was masked by the one before it, and none was visible in a summary statistic. They surfaced only because the controller was deployed sixteen times away from the operating point its constants were tuned at, and because its own logs were read rather than its outputs. We offer that as routine practice for anyone measuring an adaptive system: deploy it far from where it was tuned, and check that it acted before believing what it achieved.

REPRODUCIBILITY

All code, kernel configuration, logs and analysis are available at <https://github.com/Amritha902/ccn-linux-qdisc-study>. Every table and every inline figure in this paper is generated directly from the experiment logs by `src/analyse.py`; none is typed by hand.

REFERENCES

- [1] J. Gettys and K. Nichols, "Bufferbloat: Dark Buffers in the Internet," *Commun. ACM*, vol. 55, no. 1, pp. 57–65, Jan. 2012.
- [2] S. Floyd and V. Jacobson, "Random Early Detection Gateways for Congestion Avoidance," *IEEE/ACM Trans. Netw.*, vol. 1, no. 4, pp. 397–413, Aug. 1993.
- [3] S. Floyd, R. Gummadi, and S. Shenker, "Adaptive RED: An Algorithm for Increasing the Robustness of RED's Active Queue Management," ICSI Technical Report, Aug. 2001.
- [4] K. Nichols and V. Jacobson, "Controlling Queue Delay," *ACM Queue*, vol. 10, no. 5, pp. 20–34, May 2012.
- [5] K. Nichols, V. Jacobson, A. McGregor, and J. Iyengar, "Controlled Delay Active Queue Management," IETF RFC 8289, Jan. 2018.
- [6] T. Høiland-Jørgensen, P. McKenney, D. Täht, J. Gettys, and E. Dumazet, "The FlowQueue-CoDel Packet Scheduler and Active Queue Management Algorithm," IETF RFC 8290, Jan. 2018.
- [7] R. Pan, P. Natarajan, F. Baker, and G. White, "Proportional Integral Controller Enhanced (PIE): A Lightweight Control Scheme to Address the Bufferbloat Problem," IETF RFC 8033, Feb. 2017.
- [8] G. Ramakrishnan, M. Bhasi, V. Saicharan, L. Monis, S. D. Patil, and M. P. Tahiliani, "FQ-PIE Queue Discipline in the Linux Kernel: Design, Implementation and Challenges," in *Proc. IEEE LCN Symp.*, Oct. 2019.
- [9] T. Høiland-Jørgensen, D. Täht, and J. Morton, "Piece of CAKE: A Comprehensive Queue Management Solution for Home Gateways," in *Proc. IEEE LANMAN*, Jun. 2018.
- [10] E. Sharafzadeh, R. Matson, J. Tourrilhes, P. Sharma, and S. Ghorbani, "Self-Clocked Round-Robin Packet Scheduling," in *Proc. USENIX NSDI*, Apr. 2025.
- [11] F. Rodriguez et al. "DESiRED, Dynamic, Enhanced, and Smart iRED: A P4-AQM with Deep Reinforcement Learning and In-band Network Telemetry," *Computer Networks*, vol. 244, 2024.
- [12] S. R. Pokhrel, D. Satish, J. Kua, and A. Walid, "Distilling Large Language Models for Network Active Queue Management," *IEEE Trans. Netw.*, 2025.
- [13] M. Dery, O. Krupnik, and I. Keslassy, "QueuePilot: Reviving Small Buffers With a Learned AQM Policy," in *Proc. IEEE INFOCOM*, 2023.
- [14] M. P. Toopchinezhad and M. Ahmadi, "Machine Learning Approaches for Active Queue Management: A Survey, Taxonomy, and Future Directions," *Computer Networks*, vol. 262, May 2025.
- [15] J. Ye and K.-C. Leung, "Adaptive and Stable Delay Control for Combating Bufferbloat: Theory and Algorithms," *IEEE Systems Journal*, vol. 14, no. 1, pp. 1285–1296, Mar. 2020.
- [16] B. Briscoe, K. De Schepper, M. Bagnulo, and G. White, "Low Latency, Low Loss, and Scalable Throughput (L4S) Internet Service: Architecture," IETF RFC 9330, Jan. 2023.
- [17] K. De Schepper, B. Briscoe, and G. White, "Dual-Queue Coupled Active Queue Management (AQM) for Low Latency, Low Loss, and Scalable Throughput (L4S)," IETF RFC 9332, Jan. 2023.
- [18] S. Ha, I. Rhee, and L. Xu, "CUBIC: A New TCP-Friendly High-Speed TCP Variant," *ACM SIGOPS Oper. Syst. Rev.*, vol. 42, no. 5, pp. 64–74, 2008.
- [19] V. S. Borkar, "Stochastic Approximation with Two Time Scales," *Syst. Control Lett.*, vol. 29, no. 5, pp. 291–294, 1997.

- [20] R. Jain, D.-M. Chiu, and W. Hawe, "A Quantitative Measure of Fairness and Discrimination for Resource Allocation in Shared Computer Systems," DEC Research Report TR-301, Sep. 1984.
- [21] M. A. M. Vieira et al. "Fast Packet Processing with eBPF and XDP: Concepts, Code, Challenges, and Applications," *ACM Comput. Surv.*, vol. 53, no. 1, Article 16, 2020.
- [22] F. Baker and G. Fairhurst, "IETF Recommendations Regarding Active Queue Management," IETF RFC 7567, Jul. 2015.